

Reading and checking short Python programs

Can a program run successfully and still answer the wrong question?

By the end, you should be able to...

Trace assignment, a loop and a function on a short sequence.

Explain what a function calculates in biological language.

Use small tests to detect a denominator or boundary error.

Reconnect with the biology

GC fraction counts G and C bases among the bases your denominator actually includes.

Think silently -> explain to a neighbour -> check the measurement.

Value, name, update

```
count = 0  
count = count + 1  
print(count)
```

A variable is a name associated with a value.

The equals sign in Python assigns; it does not assert a mathematical identity.

Run the next line using the current value, not the previous one.

Lists and slices have boundaries

```
seq = "ACGTAC"  
print(seq[1:4])
```

Python starts list indices at zero.

A slice includes its start but excludes its end.

For `seq = ACGTAC`, `seq[1:4]` is `CGT`.

A loop repeats a small operation

```
count = 0
for base in "AGC":
    if base in "GC":
        count += 1
```

Start with count = 0.

For each base, ask whether it is G or C.

Increase the count when the condition is true.

A function has a contract

01

State its input, output and assumptions.

>

02

Decide what an unknown base should do to the denominator.

>

03

Decide what an empty sequence should return or reject.

Trace before you execute

Base	GC?	Running count
A	No	0
G	Yes	1
C	Yes	2

For AGC, the running counts are 0, 1, 2.

The numerator is 2 and the denominator is 3.

A trace table makes a hidden assumption visible.

A deliberate bug: the wrong denominator

```
seq = "GCNN"  
wrong_gc = sum(b in "GC" for  
b in seq) / len(seq)
```

Here N means unknown and should be excluded by the stated contract.

The numerator ignores N but the denominator below includes it.

The code runs; the biological definition is violated.

Tests should include awkward inputs

Input	Expected
GCGC	1.0
ATAT	0.0
GCNN	1.0
NN / empty	Undefined

All GC bases should give 1.0. All AT bases should give 0.0.

GCNN should give 1.0 under our known-base definition.

An empty or all-N sequence should be reported as undefined, not 0%.

A traceback is a location clue

```
NameError: name 'sequence' is  
not defined  
# Check spelling and  
execution order.
```

Read the last line: what kind of error occurred?

Read upward to the line in your notebook that triggered it.

A failed test can be more informative than a clean run.

Worked example: repair without rewriting

The function should calculate GC among A/C/G/T positions only.

Predict the result and one limitation before calculating.

Work through the reasoning

Make known = [b for b in seq.upper() if b in "ACGT"].

Divide the GC count by len(known), not len(seq).

Return None when known is empty; check GCNN and NN as controls.

Pause: make a defensible claim

A function returns 0.5 for GCNN. Under our contract, locate the error and predict the correct result.

First answer alone. Then compare the evidence, not just the conclusion.

A defensible response

The denominator includes the two unknown bases. There are two GC bases among two known positions, so the fraction is 1.0.

Today's practical

W03: trace four inputs, compare a supplied buggy and corrected function, and explain one test in words. No program from scratch is required.

Minutes	Action
0-10	Predict + set up
10-30	Run + inspect + check
30-40	Explain + compare
40-45	Save + submit / exit

Five terms to use precisely

1 variable

2 index

3 loop

4 function

5 test

Exit ticket

What did we actually measure or calculate?

What is one assumption or limitation of that result?

What check would you perform before sharing it?

Go further only after the core is clear

Optional: write a new test for lower-case letters and explain why it matters.

Not required for the core grade. No advanced model training is expected.

Read the evidence, not only the slide

S05 Python 3 tutorial: introduction and control flow.